
Name: Artem Peychev
CYBR437: Secure Coding
Assignment: Lab 2 - GDB Primer
Due Date: October 7th, 2026 at 11:59 PM

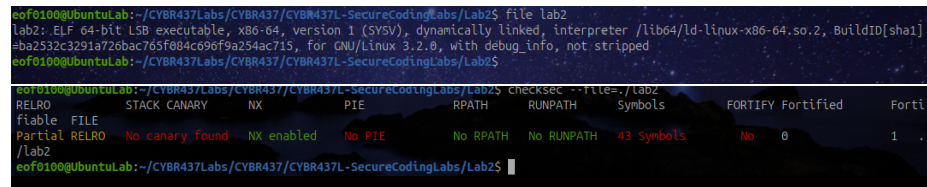
Question 1: What type of file is this, and what kind of security does it have?

It is an ELF 64 bit LSB executable that includes debugging information and is not stripped. The checksec output shows that it has Partial RELRO, NX enabled, no stack canary, and no PIE. This means the program has protections enabled, like the NX or no execute and partial relocation. The program does not use PIE or canary protection.

Screenshot below shows I used checksec to check type of security protections the file has enabled.

GDB / Linux Command(s):

```
file lab2
file --file=./lab2
```



```
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab2$ file lab2
lab2: ELF 64-bit LSB executable, x86_64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]
=ba2532c3291a720bac765f084c690f9a254ac715, for GNU/Linux 3.2.0, with debug_info, not stripped
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab2$
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab2$ checksec --file=./lab2
RELRO      STACK CANARY NX      PIE       RPATH      RUNPATH      Symbols      FORTIFY Fortified Forti
fiable FILE
Partial RELRO No canary found NX enabled No PIE      No RPATH     No RUNPATH   43 Symbols   No    0      1
./lab2
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab2$
```

Question 2: What are the files that make up this binary, and which one contains main?

The file that contains main is cscd437lab2main.c. The cybr437lab2.c contains functions used by main, cybr437lab2.h is a header file, and finally lab2 is a compiled executable.

Question 3: What are the type(s) and name(s) of parameter(s) being passed to main?

The parameters passed to main are argc (int), and argv (pointer to char). Argc stores the number of command line arguments while argv stores the arguments themselves.

GDB Command(s):

```
ptype main
```

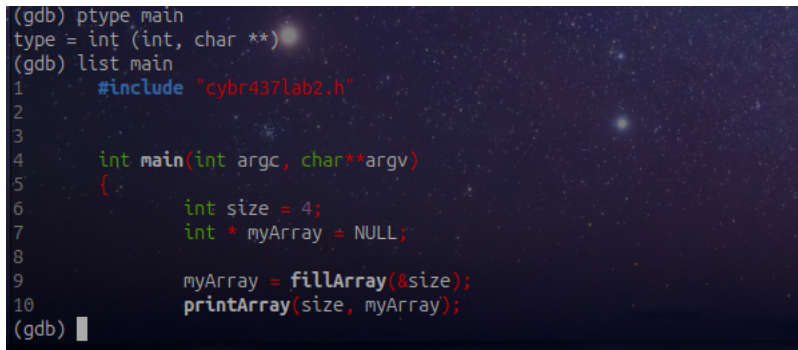
A screenshot of a GDB terminal window. The background is dark with a starry pattern. The text is white and green. The prompt is (gdb). The user has entered 'ptype main'. The output shows 'type = int (int, char **)' and '(gdb) list main'. The source code for main is listed, starting with line 1: '#include "cybr437lab2.h"', line 4: 'int main(int argc, char**argv)', line 6: '{', line 7: ' int size = 4;', line 8: ' int * myArray = NULL;', line 9: ' myArray = fillArray(&size);', line 10: ' printArray(size, myArray);', and line 11: '}'.

Figure 1: Question 3 Screenshot

Question 4: Set a breakpoint on each function and display the breakpoints.

GDB Command(s):

```
info functions
break main
break cleanUp
break *fillArray
break printArray
info breakpoints
```

Question 5: Begin running the program. How many arguments are passed to main?

One argument is passed argc = 1.

GDB Command(s):

```
run
print argc
p argc
```

```

70:         printArray(size, myArray);
(gdb) info functions
All defined functions:

File cscd437lab2main.c:
4:     int main(int, char **);

File cybr437lab2.c:
74:     int cleanUp(int **);
5:     int *fillArray(int *);
57:     void printArray(int, int *);

Non-debugging symbols:
0x0000000000401000 _init
0x0000000000401030 free@plt
0x0000000000401090 puts@plt
0x00000000004010a0 printf@plt
0x00000000004010b0 calloc@plt
0x00000000004010c0 __isoc99_scanf@plt
0x00000000004010d0 _start
0x0000000000401100 _dl_relocate_static_pie
0x0000000000401110 deregister_tm_clones
0x0000000000401140 register_tm_clones
0x0000000000401180 __do_global_ctors_aux
0x00000000004011b0 frame_dummy
0x00000000004014b4 _fini
(gdb) break main
Breakpoint 1 at 0x4011c9: file cscd437lab2main.c, line 6.
(gdb) break cleanUp
Breakpoint 2 at 0x401490: file cybr437lab2.c, line 76.
(gdb) break *fillArray
Breakpoint 3 at 0x40120f: file cybr437lab2.c, line 6.
(gdb) break printArray
Breakpoint 4 at 0x4013de: file cybr437lab2.c, line 60.
(gdb)

```

Figure 2: Question 4 Screenshot

```

(gdb) info breakpoints
Num   Type             Disp Enb Address            What
1     breakpoint      keep y   0x00000000004011c9 in main at cscd437lab2main.c:6
2     breakpoint      keep y   0x0000000000401490 in cleanUp at cybr437lab2.c:76
3     breakpoint      keep y   0x000000000040120f in fillArray at cybr437lab2.c:6
4     breakpoint      keep y   0x00000000004013de in printArray at cybr437lab2.c:60
(gdb)

```

Figure 3: Question 4 Screenshot

```
(gdb) run
Starting program: /home/eof0100/CYBR437Labs/CYBR437L-SecureCodingLabs/Lab2/lab2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading 53.04 K separate debug info for system-supplied DSO at 0x7ffff7fde000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=1, argv=0x7ffff7fde000) at cscd437lab2main.c:6
6      int size = 4;
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/eof0100/CYBR437Labs/CYBR437L-SecureCodingLabs/Lab2/lab2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=1, argv=0x7ffff7fde000) at cscd437lab2main.c:6
6      int size = 4;
(gdb) print argc
$1 = 1
(gdb) p argc
$2 = 1
(gdb) |
```

Figure 4: Question 5 Screenshot

Question 6: Step twice and display the contents of the constant and two variables in main.

Stepping twice the size was 4, myArray was 0x0 and argc is 1.

GDB Command(s):

```
next
next
print size
print myArray
print argc
```

Question 7: Step into the first function called in main. What is the type and name of the constant? Where is the constant declared?

The constant is called MAX and it is a const int type which is declared globally outside of fillArray function.

GDB Command(s):

```

(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/eof0100/CYBR437Labs/CYBR437L-SecureCodingLabs/Lab2/lab2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=1, argv=0x7ffffffdc38) at cscd437lab2main.c:6
6      int size = 4;
(gdb) print argc
$1 = 1
(gdb) p argv
$2 = 1
(gdb) next
7      int * myArray = NULL;
(gdb) next
9      myArray = fillArray(&size);
(gdb) print size
$3 = 4
(gdb) print myArray
$4 = (int *) 0x0
(gdb) print argc
$5 = 1

```

Figure 5: Question 6 Screenshot

```

step
list
info locals

```

```

(gdb) step
Breakpoint 3, fillArray (size=0x2600000) at cybr437lab2.c:6
6      {
(gdb) list
1      #include "cybr437lab2.h"
2
3      const int MAX = 9;
4
5      int * fillArray(int *size)
6      {
7          int x=0, num = 0;
8          int * array = (int*)calloc(MAX, sizeof(int));
9
10         printf("Please enter up to %d positive (>0) integers. To quit early enter -99\n\n", MAX);
(gdb) info locals
x = 0
num = 32767
array = 0xb700000000
(gdb) █

```

Figure 6: Question 7 Screenshot

Question 8: Explain the difference between printing var and var[x].

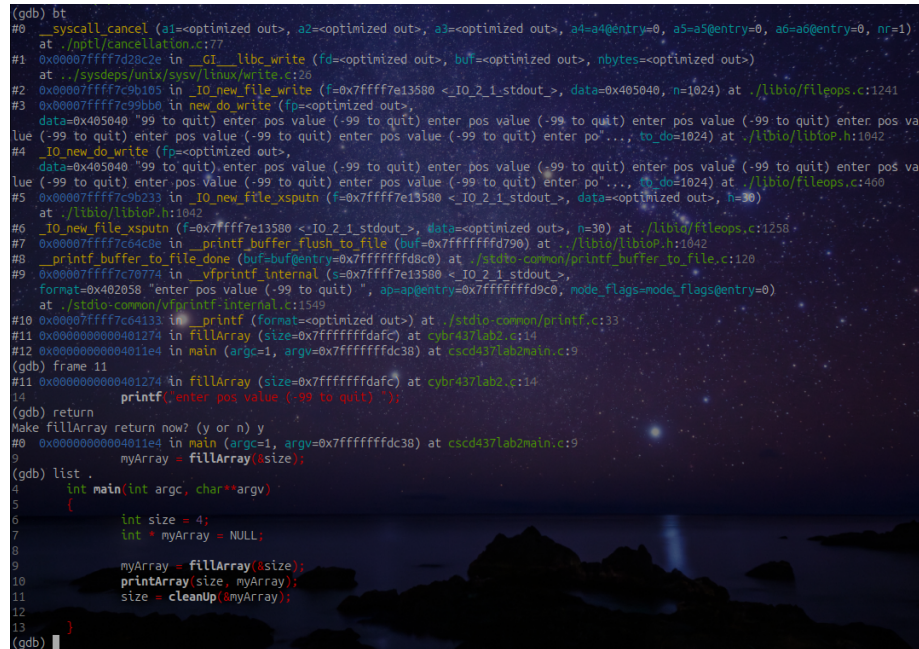
Printing the array prints out the memory address stored at the pointer location but the array[x] prints out the actual value stored at index x.

Question 9: Instead of continuing this function, return to main and print the current line.

I used bt to find fillArray which was at stack frame 11, then switched to it using frame 11 command, used return to return to main, then list . to display the current line.

GDB Command(s):

```
bt
frame 11
return
list .
```



```
(gdb) bt
#0  __syscall_cancel (a1=<optimized out>, a2=<optimized out>, a3=<optimized out>, a4=a4@entry=0, a5=a5@entry=0, a6=a6@entry=0, nr=1)
    at ./optlib/cancellation.c:77
#1  0x00007fffffd28c2e in __GI___libc_write (fd=<optimized out>, buf=<optimized out>, nbytes=<optimized out>)
    at ../sysdeps/unix/sysv/linux/write.c:26
#2  0x00007fffffc9b185 in _IO_new_file_write (f=0x7fffff7e13580 <_IO_2_1_stdout_>, data=0x405040, n=1024) at ./libio/fileops.c:1241
#3  0x00007fffffc99bb0 in new_do_write (fp=<optimized out>,
    data=0x405040 '99 to quit) enter pos value (-99 to quit) enter pos value (-99 to quit) enter pos value (-99 to quit) enter pos va
    lue (-99 to quit) enter pos value (-99 to quit) enter pos value (-99 to quit) enter po..., to_do=1024) at ./libio/libioP.h:1042
#4  _IO_new_do_write (fp=<optimized out>,
    data=0x405040 '99 to quit) enter pos value (-99 to quit) enter pos value (-99 to quit) enter pos value (-99 to quit) enter pos va
    lue (-99 to quit) enter pos value (-99 to quit) enter pos value (-99 to quit) enter po..., to_do=1024) at ./libio/fileops.c:1460
#5  0x00007fffffc9b233 in _IO_new_file_xsputn (f=0x7fffff7e13580 <_IO_2_1_stdout_>, data=<optimized out>, n=30)
    at ./libio/libioP.h:1042
#6  _IO_new_file_xsputn (f=0x7fffff7e13580 <_IO_2_1_stdout_>, data=<optimized out>, n=30) at ./libio/fileops.c:1258
#7  0x00007fffffc64c8e in __printf_buffer_flush_to_file (buf=0x7fffff7d790) at ./libio/libioP.h:1042
#8  __printf_buffer_to_file_done (buf=buf@entry=0x7fffff7d8c0) at ./stdio-common/printf_buffer_to_file.c:120
#9  0x00007fffffc70774 in __vfprintf_internal (s=0x7fffff7e13580 <_IO_2_1_stdout_>,
    format=0x402058 "enter pos value (-99 to quit) ", ap=ap@entry=0x7fffff7d9c0, mode_flags=mode_flags@entry=0)
    at ./stdio-common/vfprintf_internal.c:1349
#10  0x00007fffffc64133 in __printf (format=<optimized out>) at ./stdio-common/printf.c:33
#11  0x0000000000401274 in fillArray (size=0x7fffff7dafa0) at cybr437lab2.c:14
#12  0x00000000004011e4 in main (argc=1, argv=0x7fffff7d9c0) at cscd437lab2main.c:9
(gdb) frame 11
#11  0x0000000000401274 in fillArray (size=0x7fffff7dafa0) at cybr437lab2.c:14
13     printf("enter pos value (-99 to quit) ");
(gdb) return
Make fillArray return now? (y or n) y
#0  0x00000000004011e4 in main (argc=1, argv=0x7fffff7d9c0) at cscd437lab2main.c:9
9     myArray = fillArray(4size);
(gdb) list .
4     int main(int argc, char**argv)
5     {
6
7         int size = 4;
8         int * myArray = NULL;
9
10        myArray = fillArray(4size);
11        printArray(size, myArray);
12        size = cleanUp(&myArray);
13    }
```

Figure 7: Question 9 Screenshot

Question 10: Step into the second function called from main. What is the name and starting value of the counting variable?

As you can see from the screenshot below the name of the starting value of the counting variable is called x and its starting value is 0.

GDB Command(s):

```
next
step
info locals
print x
```



```
(gdb) next
19      printArray(size, myArray);
(gdb) step
Breakpoint 4, printArray (size=4, myArray=0x400) at cybr437lab2.c:60
60      if(size != 0)
(gdb) list
55
56
57      void printArray(int size, int * myArray)
58      {
59          int x;
60          if(size != 0)
61          {
62              printf("[ ");
63              for(x = 0; x < size - 1; x++)
64                  printf("%d, ", myArray[x]);
(gdb) info locals
x = 0
(gdb) print x
$26 = 0
(gdb)
```

Figure 8: Question 10 Screenshot

Question 11: Use the disassemble command in GDB to display the assembly code for the function. What does the output show, and how does it correlate with the C source code?

The disassemble command shows assembly code for printArray() function. The assembly correlates to the C code by showing the instruction set for the if statement, for loop, accessing array at myArray[x], increasing x, and calling printf() statement.

GDB Command(s):

```
disassemble printArray
```

```

Dump of assembler code for function printArray:
0x00000000004013cb <+0>:    endbr64
0x00000000004013cf <+4>:    push    %rbx
0x00000000004013d0 <+5>:    mov     %rsp,%rbp
0x00000000004013d3 <+8>:    sub     $0x20,%rsp
0x00000000004013d7 <+12>:   mov     %edi,-0x14(%rbp)
0x00000000004013da <+15>:   mov     %rsi,-0x20(%rbp)
=> 0x00000000004013de <+19>:   cmpl    $0x0,-0x14(%rbp)
0x00000000004013e2 <+23>:   je      0x40146e <printArray+163>
0x00000000004013e8 <+29>:   lea     0xcbc(%rip),%rax    # 0x4020ab
0x00000000004013ef <+36>:   mov     %rax,%rdi
0x00000000004013f2 <+39>:   mov     $0x0,%eax
0x00000000004013f7 <+44>:   call    0x4010a0 <printf@plt>
0x00000000004013fc <+49>:   movl    $0x0,-0x4(%rbp)
0x0000000000401403 <+56>:   jmp     0x401435 <printArray+106>
0x0000000000401405 <+58>:   mov     -0x4(%rbp),%eax
0x0000000000401408 <+61>:   cltq
0x000000000040140a <+63>:   lea     0x0(%rax,%rax,1),%rdx
0x0000000000401412 <+71>:   mov     -0x20(%rbp),%rax
0x0000000000401416 <+75>:   addq    %rdx,%rax
0x0000000000401419 <+78>:   mov     (%rax),%eax
0x000000000040141b <+80>:   mov     %eax,%esi
0x000000000040141d <+82>:   lea     0xc8a(%rip),%rax    # 0x4020ae
0x0000000000401424 <+89>:   mov     %rax,%rdi
0x0000000000401427 <+92>:   mov     $0x0,%eax
0x000000000040142c <+97>:   call    0x4010a0 <printf@plt>
0x0000000000401431 <+102>:  addl    $0x1,-0x4(%rbp)
0x0000000000401435 <+106>:  mov     -0x14(%rbp),%eax
0x0000000000401438 <+109>:  sub     $0x1,%eax
0x000000000040143b <+112>:  cmp     %eax,-0x4(%rbp)
0x000000000040143e <+115>:  jl      0x401405 <printArray+58>
0x0000000000401440 <+117>:  mov     -0x14(%rbp),%eax
0x0000000000401443 <+120>:  cltq
0x0000000000401445 <+122>:  shl     $0x2,%rax
0x0000000000401449 <+126>:  lea     -0x4(%rax),%rdx
0x000000000040144d <+130>:  mov     -0x20(%rbp),%rax
0x0000000000401451 <+134>:  addq    %rdx,%rax
0x0000000000401454 <+137>:  mov     (%rax),%eax
0x0000000000401456 <+139>:  mov     %eax,%esi
0x0000000000401458 <+141>:  lea     0xc54(%rip),%rax    # 0x4020b3
0x000000000040145f <+148>:  mov     %rax,%rdi
0x0000000000401462 <+151>:  mov     $0x0,%eax
0x0000000000401467 <+156>:  call    0x4010a0 <printf@plt>
0x000000000040146c <+161>:  jmp     0x40147d <printArray+178>
0x000000000040146e <+163>:  lea     0xc44(%rip),%rax    # 0x4020b9
--Type <RET> for more, q to quit, c to continue without paging--

```

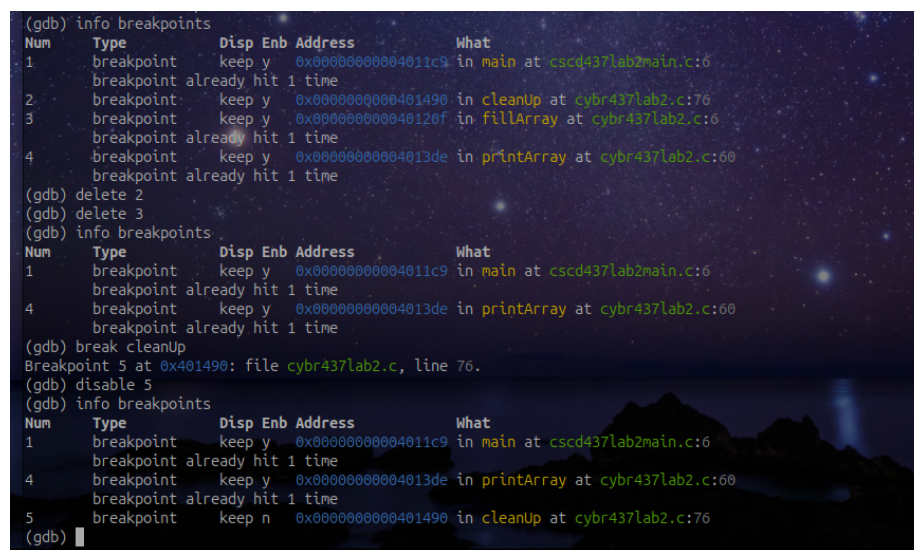
Figure 9: Question 11 Screenshot

Question 12: Delete the breakpoint for the first function called in main and disable the breakpoint for the last function called.

First, I used `info breakpoints` to view all of my breakpoints. The first function called in main is `fillArray`, so I deleted its breakpoint. The last function called in main is `cleanUp`, so I disabled its breakpoint. At first I accidentally deleted `cleanUp` but then I added it to the breakpoint then disabled it, finally, verifying everything is correct with `info breakpoints`.

GDB Command(s):

```
info breakpoints
delete 2
delete 3
info breakpoints
break cleanUp
disable 5
info breakpoints
```



```
(gdb) info breakpoints
Num   Type             Disp Enb Address            What
1     breakpoint       keep y   0x00000000004011c9 in main at cscd437lab2main.c:6
      breakpoint already hit 1 time
2     breakpoint       keep y   0x0000000000401490 in cleanUp at cybr437lab2.c:76
3     breakpoint       keep y   0x000000000040120f in fillArray at cybr437lab2.c:6
      breakpoint already hit 1 time
4     breakpoint       keep y   0x00000000004013de in printArray at cybr437lab2.c:60
      breakpoint already hit 1 time
(gdb) delete 2
(gdb) delete 3
(gdb) info breakpoints
Num   Type             Disp Enb Address            What
1     breakpoint       keep y   0x00000000004011c9 in main at cscd437lab2main.c:6
      breakpoint already hit 1 time
4     breakpoint       keep y   0x00000000004013de in printArray at cybr437lab2.c:60
      breakpoint already hit 1 time
(gdb) break cleanUp
Breakpoint 5 at 0x401490: file cybr437lab2.c, line 76.
(gdb) disable 5
(gdb) info breakpoints
Num   Type             Disp Enb Address            What
1     breakpoint       keep y   0x00000000004011c9 in main at cscd437lab2main.c:6
      breakpoint already hit 1 time
4     breakpoint       keep y   0x00000000004013de in printArray at cybr437lab2.c:60
      breakpoint already hit 1 time
5     breakpoint       keep n   0x0000000000401490 in cleanUp at cybr437lab2.c:76
(gdb)
```

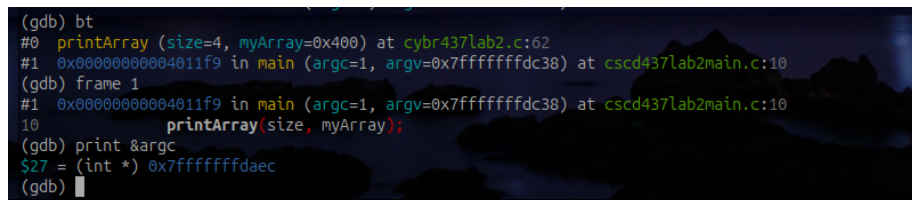
Figure 10: Question 12 Screenshot

Question 13: Without starting over, AKA your current location in a function that is not main, print the memory location of the first variable passed to main.

Using bt command to find that main was located in frame 1, switched to frame 1, and then used &argc to display the memory address of the first parameter that is passed to main which was 0x7ffffffdaec.

GDB Command(s):

```
bt
frame 1
print &argc
```



```
(gdb) bt
#0  printArray (size=4, myArray=0x400) at cybr437lab2.c:62
#1  0x00000000004011f9 in main (argc=1, argv=0x7ffffffdc38) at cscd437lab2main.c:10
(gdb) frame 1
#1  0x00000000004011f9 in main (argc=1, argv=0x7ffffffdc38) at cscd437lab2main.c:10
10      printArray(size, myArray);
(gdb) print &argc
$27 = (int *) 0x7ffffffdaec
(gdb)
```

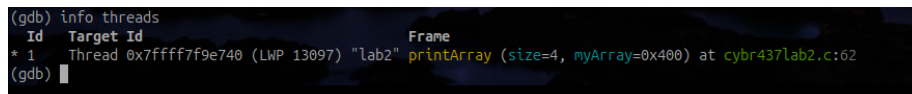
Figure 11: Question 13 Screenshot

Question 14: Print the information on the current running threads. How many threads are running?

Using info threads command displayed active threads of 1 as shown from the screenshot below.

GDB Command(s):

```
info threads
```



```
(gdb) info threads
  Id   Target Id         Frame
* 1    Thread 0x7ffff7f9e740 (LWP 13097) "lab2" printArray (size=4, myArray=0x400) at cybr437lab2.c:62
(gdb)
```

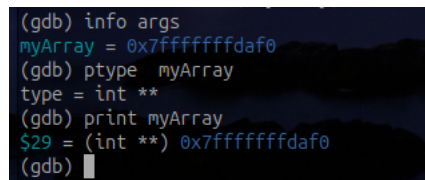
Figure 12: Question 14 Screenshot

Question 15: Enable the breakpoint for the third function called by main. What is the type, name, and memory location of the variable passed to it?

The var passed to cleanUp() function is myArray. Its type is int ** and its memory location is 0x7fffffffda0.

GDB Command(s):

```
enable 5
continue
info args
ptype myArray
print myArray
```



```
(gdb) info args
myArray = 0x7fffffffda0
(gdb) ptype myArray
type = int **
(gdb) print myArray
$29 = (int **) 0x7fffffffda0
(gdb) |
```

Figure 13: Question 15 Screenshot
